

Universidad Rafael Landívar

De Quetzaltenango



Curso: Manejo e Implementación de Archivos

Catedrático: Rojas Vásquez Manuel Gustavo

Entrega: Proyecto 1

documentación

<https://github.com/AlexisAlv30/Proyecto1-ManejoArchivos>

Nombre: Davis José Alexis Garcia López

Carné: 1554725

Quetzaltenango 16 de septiembre de 2026

Introducción

El presente informe técnico expone el diseño, desarrollo e implementación de una aplicación de escritorio orientada a la gestión y persistencia segura de configuraciones de usuario. En el ámbito del desarrollo de software, la correcta administración de archivos en el almacenamiento secundario constituye un pilar fundamental para garantizar la integridad, confiabilidad y estabilidad de los datos ante cualquier eventualidad. A lo largo de este documento, se detalla la justificación técnica detrás de la elección del formato CSV frente a alternativas como JSON, evaluando criterios de rendimiento, parseo y espacio en disco. Asimismo, se explica la implementación de mecanismos avanzados de escritura atómica mediante archivos temporales (.tmp) y copias de respaldo (.bak) para prevenir la corrupción de datos ante cierres inesperados. Finalmente, se presenta la validación del sistema ante escenarios críticos de fallo, tales como la ausencia del archivo, corrupción de formato y restricción de permisos de lectura o escritura.

Comparación de Formatos

Para la persistencia de los datos de configuración de la aplicación se evaluaron dos alternativas de almacenamiento: CSV (Comma-Separated Values / Delimitado por tuberías |) y JSON (JavaScript Object Notation).

Criterio Técnico	Formato Elegido: CSV (con)	Alternativa Descartada: JSON
Tamaño en Disco	Muy Bajo: Guarda los valores en una sola línea plana sin sintaxis repetitiva.	Moderado: Requiere llaves, comillas y nombres de propiedades repetidos por cada objeto.
Legibilidad y Parseo	Sencillo: Lectura directa secuencial usando delimitadores (DictReader)	Estructurado: Fácil de leer pero requiere procesar estructuras jerárquicas completas en memoria.
Robustez ante Corrupción	Alta: Al ser una sola fila plana con delimitador , un fallo se detecta y aísla de inmediato.	Sensible: Un solo corchete o coma faltante corrompe todo el árbol del documento.
Manejo de Caracteres	Excelente: Control directo mediante codificación explícita UTF-8.	Excelente: Compatible de forma nativa con Unicode.

Justificación

La selección del formato CSV delimitado por tuberías (|) responde a la necesidad de mantener un esquema de almacenamiento liviano, determinista y de alto rendimiento para registros de configuración planos. A diferencia de formatos jerárquicos como JSON o XML que introducen sobrecarga sintáctica mediante llaves, comillas dobles y etiquetas repetitivas, el formato CSV reduce al mínimo la huella en disco y simplifica el proceso de parseo secuencial mediante DictReader. La elección de la tubería (|) como separador previene conflictos de análisis (parsing errors) cuando los campos contienen comas o espacios, garantizando una lectura directa y rápida. Además, al procesarse como un flujo de texto plano con codificación UTF-8 explícita, se facilita la detección inmediata de líneas corruptas y se reduce drásticamente la complejidad computacional requerida para deserializar los datos al arrancar la aplicación.

Implementación de Escritura Segura

Para garantizar la integridad física de los datos ante cierres abruptos o fallas del sistema, se implementó un mecanismo de **guardado atómico** combinado con **copias de respaldo**:

1. **Generación de Respaldo (.bak):** Antes de modificar la configuración existente, el sistema copia el archivo original config.csv a config.csv.bak.
2. **Escritura Atómica en Archivo Temporal (.tmp):** Los nuevos datos se escriben en config.csv.tmp. Se aplican las funciones flush() y os.fsync() para forzar el vaciado del buffer de memoria directamente al disco físico.
3. **Reemplazo Atómico:** Se utiliza os.replace() para sustituir el archivo temporal por el definitivo en una sola instrucción a nivel de Sistema Operativo.

Fragmento de código clave (Escritura atómica y respaldo)

```
if os.path.exists(self.filepath):
```

```
    shutil.copy2(self.filepath, self.bak_filepath)
```

```
with open(self.tmp_filepath, mode="w", newline="", encoding="utf-8") as tmp_file:
```

```
    writer = csv.DictWriter(tmp_file, fieldnames=self.FIELDS, delimiter="|")
```

```
    writer.writeheader()
```

```
    writer.writerow(config_dict)
```

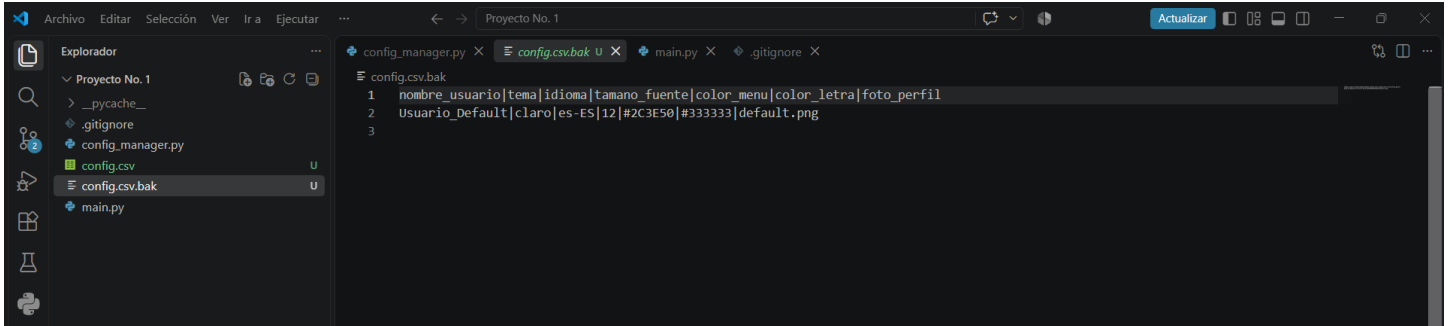
```
    tmp_file.flush()
```

```
    os.fsync(tmp_file.fileno())
```

```
os.replace(self.tmp_filepath, self.filepath)
```

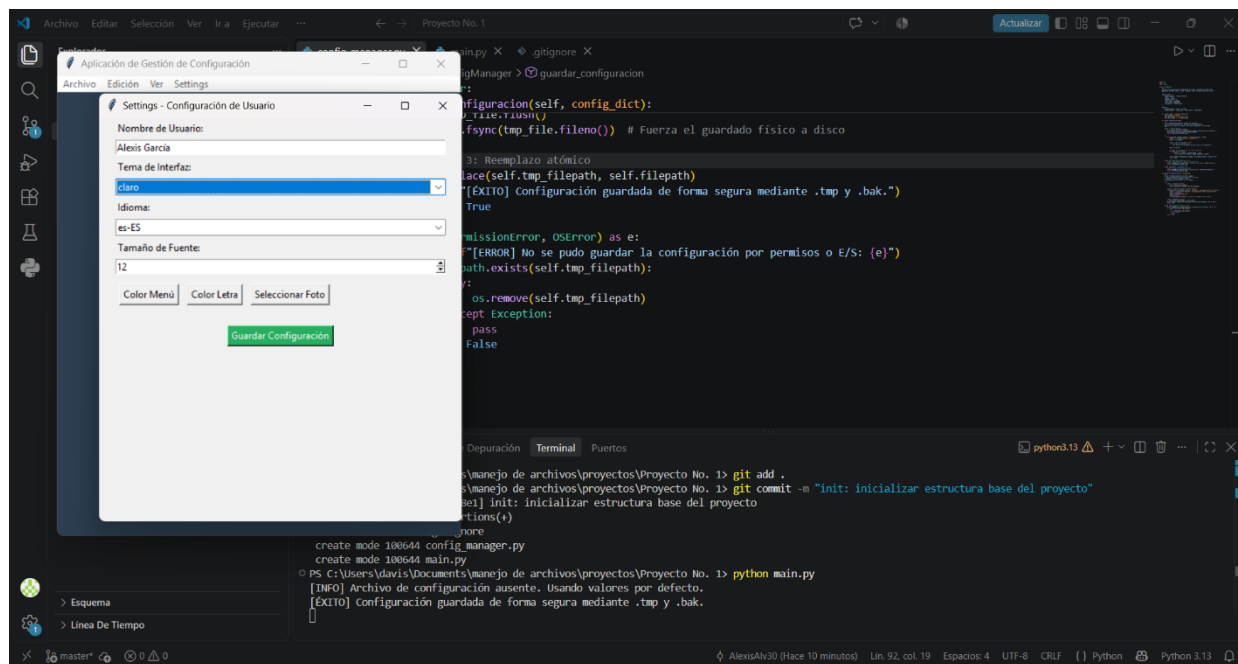
Evidencia de Generación de Respaldo (.bak)

Para verificar que el sistema conserva una copia intacta del estado previo antes de cualquier modificación, se generó el archivo de respaldo config.csv.bak. En la siguiente imagen se observa el árbol de directorios del proyecto con el archivo .bak creado y su contenido correspondiente a la configuración anterior:



La interfaz gráfica de usuario desarrollada en Tkinter

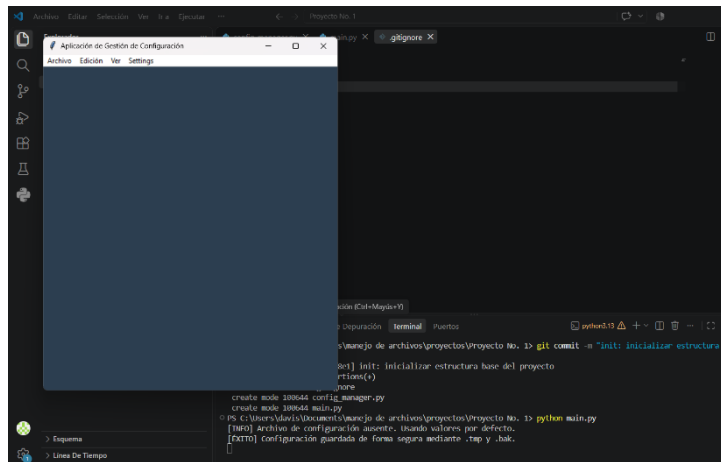
permite la interacción directa con el motor de persistencia. A continuación, se muestra la ventana emergente de *Settings* en ejecuciones reales, donde se parametrizan los valores de usuario, tema, idioma, tamaño de fuente, colores de menú/letra y la ruta de la foto de perfil:



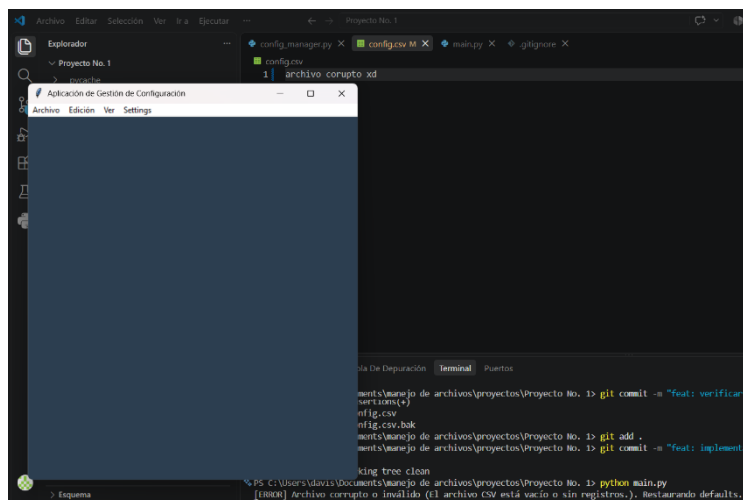
Manejo Robusto de Excepciones y Evidencias

El sistema cuenta con un control estricto de excepciones para evitar cierres inesperados (*tracebacks*):

- **Caso 1: Archivo Ausente (FileNotFoundError)**
 - *Comportamiento:* Si el sistema no detecta config.csv, genera de forma automática el archivo asignando los valores predeterminados (*defaults*).

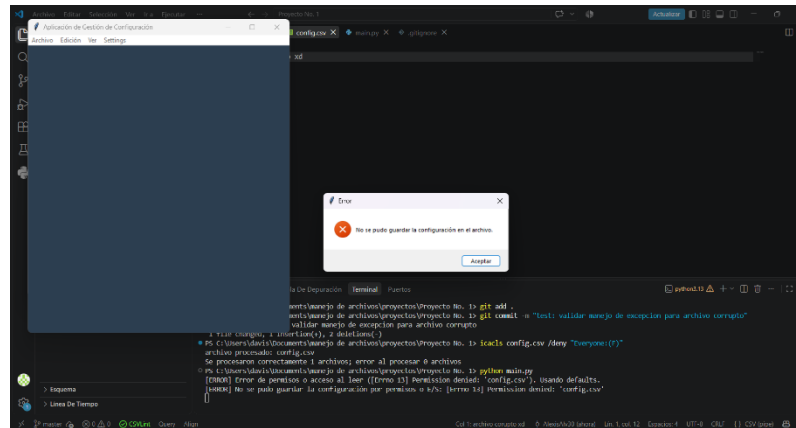


- **Caso 2: Archivo Corrupto o Formato Inválido (ValueError / csv.Error)**
 - *Comportamiento:* Si el archivo contiene datos dañados, campos incompletos o tipos de datos erróneos (ej. texto en el tamaño de fuente), el parser captura la excepción, notifica el fallo y carga la configuración por defecto.



- **Caso 3: Falta de Permisos de E/S (PermissionError)**

- *Comportamiento:* Si el archivo está marcado como "Sólo lectura" o el SO deniega el acceso de escritura, la aplicación captura la excepción PermissionError, evita la corrupción del archivo original y despliega un aviso explícito al usuario.



Evidencia de Codificación UTF-8

La lectura y escritura se fuerzan mediante `encoding='utf-8'`. A continuación se muestra un ejemplo del archivo `config.csv` generado conteniendo tildes y caracteres especiales:

```
nombre_usuario|tema|idioma|tamano_fuente|color_menu|color_letra|foto_perfil
José Alexis García|oscuro|es-ES|14|#2C3E50|#FFFFFF|C:/Imágenes/perfil.png
```